

Project 3: Checkers

Released: March 31st

Wireframes Due: Tuesday, April 7th at 5pm on Gradescope

Diagrams Due: Tuesday, April 14th at 5pm on Gradescope

Code and Report Due: Tuesday, April 21st at 5pm on Blackboard and Gradescope

Use up to three late days on this project. You may work in pairs

1 Introduction

This project will have you implementing the game Checkers in an application that can be played over the network.

2 Checkers

Checkers is a two player boardgame played by moving pieces on a chessboard. Players begin with a set layout of pieces on a chessboard and pieces move diagonally. A player wins when they capture all of their opponent's pieces, or can prevent the opposing player from moving. If neither player is capable of moving, the game ends in a draw. The rulebook to the game is attached as an appendix to this document.

3 Your work

Work for this project will be submitted as three components. Wireframes will be due on April 10th to gradescope. Class diagrams will be submitted to gradescope on April 17th. Code will be due to blackboard on April 25th and the optional "Above-and-beyond" report will be due on April 25th to gradescope. All submission times are at 5pm. You may use up to three late days on this project.

3.1 Code

Your submission should contain two zip files containing maven projects: one for the server and one for the client. At a minimum, your code must support the following behavior. **The HW5 starter code is a good starting place for this project, although it is not required**

- A Client application that allows you to connect to a server to play against another human opponent. Your code may assume that there is a running server to connect to. Users should be able to play multiple games and the correct game state should be maintained.
- Server code, which may optionally contain a GUI, will match clients against each other for play and keep track of the game state for each pairing. Server code will always be run before a client connects.

Your combined project must meet the minimum requirements below:

- a) The server should be able to support multiple pairs of clients each playing a game of connect four simultaneously.
- b) The app should correctly determine a win, loss or draw for each game and at the end of each game, there should be an option to either play again with the same player or quit.
- c) The client must use a GUI with graphical elements. The GUI should be intuitive and all user actions should be clearly labeled.
- d) The server code should print a log of all activity on the server. A GUI for the server application may be useful here, but is not required.
- e) Users must create a unique username when logging on to the server. Duplicate usernames should give an error message to the user and prompt them to enter a new name
- f) Players in a game should be able to send text messages to each other.
- g) All scenes in the app should be reachable and neither the client nor the server should freeze or crash during gameplay.

3.2 Above and Beyond

Completing the minimum requirements for this project will earn a 80%. To earn above this, you or your team must extend the minimum requirements and document these extensions in a report. Here is a non-exhaustive list of how you might extend the requirements to earn more points, roughly sorted from simplest to most complex:

- Implement a non-trivial AI to allow single-user play.
- Keep track of wins and losses for the usernames that persists even if the server and client are closed and reopened.
- Implement a username and password logon screen for server connections
- Allow users to add friends and see when they are online

If you or your team completed any Above and Beyond components, you should submit a report explaining your design decisions and how the user experience would be improved by your addition.

3.3 Best in Show

Only the top three projects for this class will receive a 100 on the project. These will be awarded on my judgement alone and there are no regrades. Top three announcements will be made in the last week in class and will be asked to give a short 10 minute presentation on their project on the final day of class May 1st. Additionally, the top three projects will have their code made available to the class.

4 Submitting your work

For the wireframes, create a pdf of your wireframe and submit the file to gradescope by April 7th at 5pm. If you are working in a team, submit as a group on gradescope and submit the team form. Check the blackboard / piazza post on this project for the link. Class diagrams for your client and server should be submitted as a pdf to gradescope by April 14th at 5pm. These must be digitally generated. Be sure to add your teammate if you have one when submitting. Once you have completed your app and your report (if applicable) you are ready to submit the zip of your project. Make sure that the submission runs with the maven command and be sure to perform a `mvn clean` before submitting. For late days, late days can be used for either the wireframes, the diagrams or the final submission, e.g. if

you use two late days on the wireframes, you can use two days on the final submission and only use two days cumulatively. Remember that you and your partner should have late days remaining if you plan to submit late.

4.1 Working in Pairs

If you plan to work in a pair, please fill out the "[Project Partners Form](#)" when you submit. The link is also available on blackboard. **Both team members need to submit the partners form.** Be sure that the gradescope submissions include both partners as a groups submission.

4.2 Academic Dishonesty / ChatGPT

A reminder from the first week of class that **ChatGPT and other AI tools are are not allowed on this project.** If there is suspicion of ChatGPT use or other forms of academic dishonesty, you will be asked to come and explain your code to me personally. If you cannot explain any line of your code, either in its function or its purpose, you will receive a zero on the assignment and a letter grade drop.

STANDARD CHECKERS RULES

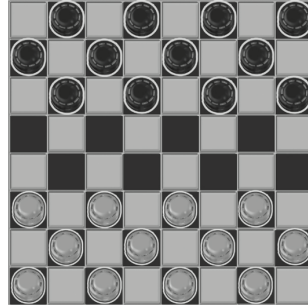
OBJECT OF THE GAME

The object of the game is to be the last player standing. A player wins the game when his opponent can no longer make a move. This happens when all his opponent's pieces have been captured or when all of his opponent's pieces are blocked in.

SETUP

Each player begins with 12 pieces placed on the game board as shown.

The board consists of 64 squares, alternating between 32 black and 32 red squares. Position it so that each player has a red square on the closest right side corner.



Set-up

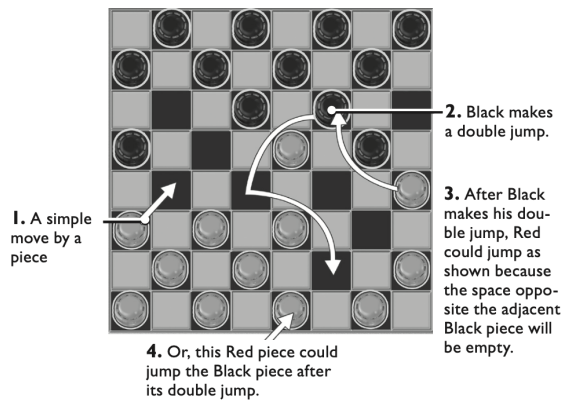
HOW TO PLAY

Black moves first. Players then alternate moves. Moves are allowed only on the black squares; pieces only move diagonally forward toward the opponent. However, Kings, as you'll see, can also move backwards. A piece may only move one square unless it is making a jump.

JUMPING

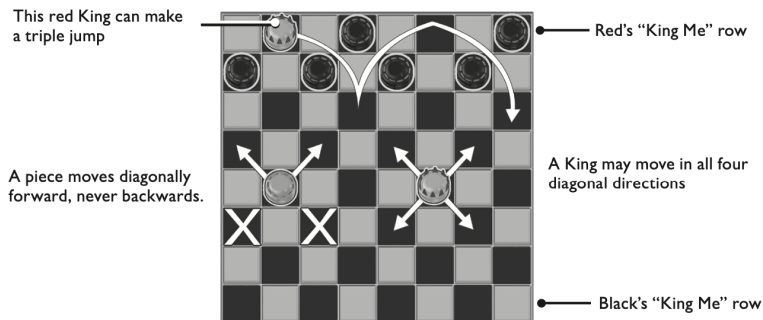
A piece makes a jump by diagonally leaping over one of the opponent's pieces and landing on the empty space on the other side (there must be an empty space to land on). Multiple jumps are allowed on a single turn. When a piece is jumped ("captured"), it is removed from the board and is now out of play.

A player must make a jump if one presents itself. This is not an option. You must jump if you can and keep jumping if you have the opportunity. If you have more than one piece that can make a jump, you can decide which one to move on your turn.



KINGS

When a piece reaches a space in the row on the opposite side of the board (the "King Me" row), its player picks up the piece and crowns it by twisting its base counter-clockwise, then sets it back on its space. It is now a King! Kings are more powerful than pieces because they can move diagonally forward and backwards. Kings may combine jumps in several directions—forward and backward—on the same turn.



WINNING

A player wins the game when the opponent cannot make a move. Usually, this is because all of the opponent's pieces have been captured, but sometimes it is because the opponent has no space to move onto (he is "blocked" from moving).